

LAPORAN PRAKTEK
SISTEM INFORMASI DATA MAHASISWA BERBASIS
PYTHON

Disusun guna memenuhi tugas mata kuliah

PEMOGRAMAN BERBASIS COMPUTER

Dosen Pengampu

Liza afriyanti M.Kom



UIN SUSKA RIAU

Disusun oleh kelompok 2:

- 1. Syifa Badriyah Rifalni (12550125418)**
- 2. Indana Nadhira (12550125417)**
- 3. Nanda Shaybila Putri (12550125203)**

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI SUKTAN SYARIF KASIM
T.A 2026/2027

KATA PENGANTAR

Puji syukur kami hantarkan kehadirat Allah SWT, yang telah melimpahkan rahmat Dan karunia-nya sehingga kami dapat menyelesaikan makalah ini sesuai dengan waktu yang telah ditentukan. Sholawat serta salam semoga selalu tercurahkan kepada junjungan kita nabi Besar Muhammad SAW, beserta keluarga dan sahabatnya. Makalah ini disusun untuk memenuhi mata kuliah Pemograman Berorientasi Objek, yang membahas mengenai *Unified Modeling Language (Uml)*, saya menyadari bahwa dalam penyusunan makalah ini masih jauh dari kata sempurna, baik dari segi bahasa maupun teknik penulisan, oleh karena itu, kami mengharapkan kritik dan saran yang membangun, khususnya bagi dosen pembimbing mata kuliah ini guna menjadi acuan bagi kami untuk lebih baik lagi dalam menyusun makalah ini. Semoga makalah ini dapat bermanfaat bagi pembaca khususnya, dan bagi masyarakat pada umumnya.

Pekanbaru, 08 Juni 2026

Penulis

DAFTAR ISI

COVER	1
KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
BAB II ANALISIS KEBUTUHAN	2
2.1 Analisis Sistem	2
2.2 Identifikasi Aktor	2
2.3 Kebutuhan Fungsional	2
2.4 Kebutuhan Non-Fungsional	3
2.5 Analisis Konsep OOP	4
BAB III	5
DESAIN SISTEM	5
3.1 Deskripsi Sistem	5
3.2 Use Case Diagram	5
3.3 Class Diagram	6
3.4 State Diagram	6
3.5 Desain Kelas	7
BAB IV	10
IMPLEMENTASI SISTEM	10
4.1 Implementasi Program	10
4.2 Implementasi LoginMixin	10
4.3 Implementasi NotificationMixin	10
4.4 Implementasi State Machine	11
4.5 Implementasi Abstract Class	12
4.6 Implementasi Inheritance	12

4.7 Implementasi Polymorphism	13
4.8 Implementasi Enkapsulasi.....	14
4.9 Implementasi Kelas Dosen	14
4.10 Implementasi Kelas Mata Kuliah.....	15
4.11 Implementasi Kelas Nilai	15
4.12 Implementasi CRUD.....	15
4.13 Implementasi Sistem Kampus.....	16
4.14 Hasil Implementasi	16
BAB V	24
PENGUJIAN SISTEM.....	24
5.1 Tujuan Pengujian	24
5.2 Pengujian Login.....	24
5.3 Pengujian Menambah Mahasiswa	24
5.4 Pengujian Menampilkan Data Mahasiswa	25
5.5 Pengujian State Machine.....	26
5.6 Pengujian Notifikasi.....	26
5.7 Pengujian Data Dosen	27
5.8 Pengujian Data Mata Kuliah.....	27
5.9 Pengujian Perhitungan Nilai	27
5.10 Rekapitulasi Pengujian.....	28
BAB VI	30
KESIMPULAN DAN SARAN	30
6.1 Kesimpulan	30
6.2 Saran	30
DAFTAR PUSTAKA.....	31

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengelolaan data mahasiswa menjadi salah satu hal yang penting dalam kegiatan akademik. Data seperti nama, NIM, program studi, dan informasi lainnya perlu disimpan dengan baik agar mudah diakses saat dibutuhkan. Namun, jika masih dilakukan secara manual, proses pengelolaan data bisa menjadi kurang praktis dan lebih rentan terhadap kesalahan.

Oleh karena itu, diperlukan sebuah sistem yang dapat membantu mengelola data mahasiswa dengan lebih mudah dan teratur. Melalui Sistem Informasi Data Mahasiswa berbasis Python, proses input, pencarian, dan pengelolaan data dapat dilakukan dengan lebih cepat dan efisien. Selain memudahkan pengguna, sistem ini juga menjadi salah satu penerapan bahasa pemrograman Python dalam menyelesaikan kebutuhan pengelolaan data.

BAB II

ANALISIS KEBUTUHAN

2.1 Analisis Sistem

Sistem Informasi Data Mahasiswa adalah aplikasi sederhana yang dibuat menggunakan Python untuk membantu mengelola data yang berkaitan dengan kegiatan akademik, seperti data mahasiswa, dosen, mata kuliah, dan nilai. Sistem ini dibuat dengan menerapkan konsep Object Oriented Programming (OOP) agar program lebih rapi, mudah dipahami, dan lebih mudah dikembangkan jika diperlukan penambahan fitur di kemudian hari. Melalui sistem ini, admin dapat menambahkan data mahasiswa, melihat data yang sudah tersimpan, mengelola nilai mahasiswa, menampilkan data dosen dan mata kuliah, serta mengirim notifikasi kepada mahasiswa. Selain itu, sistem juga memanfaatkan State Machine untuk mengatur perubahan status mahasiswa dan Abstract Base Class (ABC) sebagai salah satu penerapan konsep OOP.

2.2 Identifikasi Aktor

Pengguna yang menjalankan sistem ini adalah Admin. Beberapa tugas yang dapat dilakukan oleh Admin antara lain:

- 1) Mengelola data mahasiswa.
- 2) Mengelola data dosen.
- 3) Mengelola data mata kuliah.
- 4) Mengelola data nilai mahasiswa.
- 5) Mengirim notifikasi kepada mahasiswa.

2.3 Kebutuhan Fungsional

Fitur-fitur yang tersedia dalam sistem ini meliputi:

1. Login Sistem

Admin dapat masuk ke dalam sistem dan keluar dari sistem melalui fitur login dan logout.

- 1) Menambah Data Mahasiswa

Admin dapat memasukkan data mahasiswa baru ke dalam sistem.

2) Menampilkan Data Mahasiswa

Sistem dapat menampilkan seluruh data mahasiswa yang telah tersimpan.

3) Mengubah Status Mahasiswa

Sistem dapat memperbarui status mahasiswa sesuai kondisi yang diperlukan.

4) Mengirim Notifikasi

Admin dapat mengirimkan informasi atau pemberitahuan kepada mahasiswa.

5) Menampilkan Data Dosen

Sistem dapat menampilkan informasi mengenai dosen.

6) Menampilkan Data Mata Kuliah

Sistem dapat menampilkan daftar mata kuliah yang tersedia.

7) Menghitung Nilai Mahasiswa

Sistem dapat menghitung nilai akhir mahasiswa berdasarkan nilai tugas, UTS, dan UAS.

2.4 Kebutuhan Non-Fungsional

1) Perangkat Keras

2) Laptop atau komputer.

3) RAM minimal 4 GB.

4) Kapasitas penyimpanan minimal 1 GB.

5) Perangkat Lunak

6) Windows 10 atau Windows 11.

7) Python versi 3.10 atau yang lebih baru.

8) Visual Studio Code.

9) Command Prompt atau Terminal.

2.5 Analisis Konsep OOP

1) Abstraction

Konsep ini digunakan pada kelas Mahasiswa yang berperan sebagai kelas dasar dan menjadi acuan bagi kelas lainnya.

2) Encapsulation

Konsep ini diterapkan dengan menyembunyikan data tertentu, seperti nama dan NIM, agar tidak bisa diakses secara langsung dari luar kelas.

3) Inheritance

Konsep pewarisan digunakan pada kelas MahasiswaInformatika dan MahasiswaSistemInformasi yang dibuat berdasarkan kelas Mahasiswa.

4) Polymorphism

Konsep ini diterapkan pada method tampilkan_info(), sehingga setiap kelas dapat menampilkan informasi dengan caranya masing-masing.

5) Mixins

Mixins digunakan untuk menambahkan fitur tambahan, seperti login dan pengiriman notifikasi, tanpa mengubah fungsi utama program.

6) State Machine

State Machine digunakan untuk mengatur perubahan status mahasiswa agar lebih mudah dikelola oleh sistem.

BAB III

DESAIN SISTEM

3.1 Deskripsi Sistem

Sistem Informasi Data Mahasiswa dibuat untuk membantu pengelolaan data akademik supaya lebih mudah dan tidak dilakukan secara manual. Data yang dikelola dalam sistem ini meliputi data mahasiswa, dosen, mata kuliah, dan nilai mahasiswa. Sistem dibuat menggunakan konsep Object Oriented Programming (OOP) sehingga program menjadi lebih terstruktur dan setiap bagian memiliki tugas masing-masing.

Dengan adanya sistem ini, proses pengelolaan data menjadi lebih praktis, cepat, dan mudah dipahami. Selain itu, program juga lebih mudah dikembangkan jika suatu saat ingin menambahkan fitur baru.

3.2 Use Case Diagram

1. Aktor

Pada sistem ini hanya ada satu pengguna utama, yaitu Admin. Admin bertugas mengelola seluruh data yang ada di dalam sistem.

2. Use Case

Beberapa hal yang bisa dilakukan oleh Admin yaitu:

- Login ke sistem.
- Menambahkan data mahasiswa.
- Melihat data mahasiswa.
- Mengubah status mahasiswa.
- Mengirim notifikasi kepada mahasiswa.
- Melihat data dosen.
- Melihat data mata kuliah.
- Menghitung nilai mahasiswa.

Use Case Diagram digunakan untuk menunjukkan hubungan antara Admin dengan fitur-fitur yang tersedia pada sistem.

3.3 Class Diagram

Sistem ini terdiri dari beberapa kelas yang saling terhubung dan bekerja sama dalam menjalankan program. Setiap kelas mempunyai fungsi yang berbeda sesuai dengan kebutuhan sistem.

Kelas yang digunakan dalam sistem ini yaitu:

- LoginMixin
- NotificationMixin
- StateMachine
- Mahasiswa (Abstract Class)
- MahasiswaInformatika
- MahasiswaSistemInformasi
- Dosen
- MataKuliah
- Nilai
- Kelas
- SistemKampus

Hubungan antar kelas menunjukkan penerapan konsep OOP. Kelas Mahasiswa menjadi kelas utama yang kemudian diturunkan menjadi MahasiswaInformatika dan MahasiswaSistemInformasi. Sedangkan kelas lainnya digunakan untuk mendukung pengelolaan data yang ada dalam sistem.

3.4 State Diagram

State Diagram digunakan untuk menggambarkan perubahan status mahasiswa dalam sistem. Status mahasiswa dapat berubah dari Aktif menjadi Cuti, lalu dapat kembali lagi menjadi Aktif.

Perubahan status tersebut dilakukan menggunakan method `change_state()` yang terdapat pada kelas StateMachine.

3.5 Desain Kelas

Kelas Mahasiswa

Kelas ini digunakan untuk menyimpan data dasar mahasiswa seperti nama dan NIM.

Atribut:

- __nama
- __nim

Method:

- get_nama()
- get_nim()
- set_nama()
- tampilkan_info()

Kelas MahasiswaInformatika

Kelas ini digunakan untuk menyimpan data mahasiswa dari Program Studi Informatika.

Atribut:

- bahasa

Method:

- tampilkan_info()

Kelas MahasiswaSistemInformasi

Kelas ini digunakan untuk menyimpan data mahasiswa dari Program Studi Sistem Informasi.

Atribut:

- bidang

Method:

- tampilkan_info()

Kelas Dosen

Kelas ini digunakan untuk menyimpan informasi dosen.

Atribut:

- nama
- nidn

Method:

- tampilkan_dosen()

Kelas MataKuliah

Kelas ini digunakan untuk menyimpan data mata kuliah yang ada dalam sistem.

Atribut:

- kode
- nama
- sks

Method:

- tampilkan_matkul()

Kelas Nilai

Kelas ini digunakan untuk mengelola nilai mahasiswa.

Atribut:

- tugas
- uts
- uas

Method:

- hitung_nilai()

Kelas Kelas

Kelas ini berfungsi untuk mengatur data mahasiswa yang berada dalam satu kelas.

Atribut:

- nama_kelas
- daftar_mahasiswa

Method:

- tambah_mahasiswa()
- hapus_mahasiswa()
- tampilkan_semua()

Kelas SistemKampus

Kelas ini digunakan untuk mengelola data dosen dan mata kuliah yang ada dalam sistem.

Atribut:

- daftar_dosen
- daftar_matkul

Method:

- tambah_dosen()
- tambah_matkul()

BAB IV

IMPLEMENTASI SISTEM

4.1 Implementasi Program

Sistem Informasi Data Mahasiswa dibuat menggunakan bahasa pemrograman Python dengan menerapkan konsep Object Oriented Programming (OOP). Program ini terdiri dari beberapa kelas yang saling terhubung untuk menjalankan berbagai fitur yang ada di dalam sistem.

4.2 Implementasi LoginMixin

Kelas LoginMixin digunakan untuk menangani proses login dan logout pada sistem.

```
class LoginMixin:

    def login(self):

        print("Login berhasil")

    def logout(self):

        print("Logout berhasil")
```

Kelas ini dibuat agar fitur login dan logout bisa digunakan kembali oleh kelas lain tanpa perlu menulis kode yang sama berulang kali.

4.3 Implementasi NotificationMixin

Kelas NotificationMixin digunakan untuk mengirim pesan atau notifikasi kepada mahasiswa.

```
class NotificationMixin:

    def kirim_notifikasi(self, pesan):

        print(f"Notifikasi: {pesan}")
```

Contoh penggunaan:

```
mhs1.kirim_notifikasi(

    "Jadwal kuliah dimulai"

)
```

Output:

```
Notifikasi: Jadwal kuliah dimulai
```

Fitur ini memudahkan sistem dalam menyampaikan informasi kepada mahasiswa.

4.4 Implementasi State Machine

State Machine digunakan untuk mengatur status mahasiswa, misalnya dari aktif menjadi cuti atau sebaliknya.

```
class StateMachine:

    def __init__(self):

        self.state = "Aktif"

    def change_state(self, state_baru):

        self.state = state_baru
```

Contoh penggunaan:

```
mhs1.change_state("Cuti")
```

Output:

```
State berubah menjadi: Cuti
```

Dengan fitur ini, perubahan status mahasiswa dapat dilakukan dengan lebih mudah.

4.5 Implementasi Abstract Class

Kelas Mahasiswa dibuat sebagai Abstract Base Class (ABC) yang menjadi dasar bagi kelas mahasiswa lainnya.

```
class Mahasiswa(
    ABC,
    LoginMixin,
    NotificationMixin
)
```

Kelas ini memiliki atribut private yaitu:

- `__nama`
- `__nim`

Selain itu, terdapat method abstrak yang harus dibuat kembali pada kelas turunannya.

```
@abstractmethod
def tampilkan_info(self):
    pass
```

4.6 Implementasi Inheritance

Konsep inheritance digunakan agar kelas turunan bisa menggunakan atribut dan method yang sudah ada pada kelas induknya.

MahasiswaInformatika

```
class MahasiswaInformatika(  
    Mahasiswa,  
    StateMachine  
)
```

Atribut tambahan:

- bahasa

MahasiswaSistemInformasi

```
class MahasiswaSistemInformasi(  
    Mahasiswa,  
    StateMachine  
)
```

Atribut tambahan:

- bidang

Dengan cara ini, penulisan program menjadi lebih singkat dan tidak banyak mengulang kode yang sama.

4.7 Implementasi Polymorphism

Polymorphism diterapkan pada method `tampilkan_info()`. Walaupun nama method yang digunakan sama, informasi yang ditampilkan bisa berbeda sesuai dengan jenis mahasiswa.

Contoh:

```
mhs1.tampilkan_info()
mhs2.tampilkan_info()
```

Hasil yang muncul akan menyesuaikan dengan data dari masing-masing mahasiswa.

4.8 Implementasi Enkapsulasi

Enkapsulasi diterapkan pada atribut:

```
self.__nama
self.__nim
```

Data tersebut tidak bisa diakses langsung dari luar kelas. Untuk melihat atau mengubah data, digunakan method:

- `get_nama()`
- `get_nim()`
- `set_nama()`

Cara ini membuat data lebih aman dan terkontrol.

4.9 Implementasi Kelas Dosen

Kelas Dosen digunakan untuk menyimpan informasi dosen yang ada di dalam sistem.

Atribut:

- `nama`
- `nidn`

Method:

- `tampilkan_dosen()`

Contoh output:

```
Nama Dosen : Budi Santoso
NIDN      : 112233
```

4.10 Implementasi Kelas Mata Kuliah

Kelas MataKuliah digunakan untuk menyimpan informasi mengenai mata kuliah.

Atribut:

- kode
- nama
- sks

Method:

- tampilkan_matkul()

Kelas ini membantu dalam mengelola data mata kuliah yang tersedia.

4.11 Implementasi Kelas Nilai

Kelas Nilai digunakan untuk menghitung nilai akhir mahasiswa berdasarkan nilai tugas, UTS, dan UAS.

Rumus yang digunakan:

$$\text{Nilai Akhir} = (\text{Tugas} + \text{UTS} + \text{UAS}) / 3$$

Contoh:

Nilai(80,85,90)

Hasil:

85.0

Dengan fitur ini, perhitungan nilai akhir dapat dilakukan secara otomatis.

4.12 Implementasi CRUD

Konsep CRUD diterapkan pada kelas Kelas untuk mengelola data mahasiswa.

Create

- tambah_mahasiswa()

Read

- tampilkan_semua()

Delete

- hapus_mahasiswa()

Fitur ini memudahkan proses penambahan, melihat, dan menghapus data mahasiswa.

4.13 Implementasi Sistem Kampus

Kelas SistemKampus digunakan untuk mengelola data dosen dan mata kuliah yang ada dalam sistem.

Method yang digunakan yaitu:

- tambah_dosen()
- tambah_matkul()

Melalui kelas ini, data dosen dan mata kuliah dapat dikelola dengan lebih mudah.

4.14 Hasil Implementasi

Setelah program dijalankan, semua fitur dapat berfungsi dengan baik sesuai yang diharapkan. Sistem mampu menampilkan data mahasiswa, dosen, mata kuliah, nilai mahasiswa, status mahasiswa, serta notifikasi yang dibutuhkan dalam proses pengelolaan data akademik.

```
from abc import ABC, abstractmethod

# =====
# MIXIN
# =====

class LoginMixin:
    def login(self):
        print("Login berhasil")

    def logout(self):
        print("Logout berhasil")
```

```

class NotificationMixin:
    def kirim_notifikasi(self, pesan):
        print(f'Notifikasi: {pesan}')

# =====
# STATE MACHINE
# =====

class StateMachine:
    def __init__(self):
        self.state = "Aktif"

    def change_state(self, state_baru):
        self.state = state_baru
        print(f'State berubah menjadi: {self.state}')

# =====
# ABSTRACT CLASS
# =====

class Mahasiswa(ABC, LoginMixin, NotificationMixin):

    def __init__(self, nama, nim):
        # ENKAPSULASI
        self.__nama = nama
        self.__nim = nim

    # Getter
    def get_nama(self):
        return self.__nama

    def get_nim(self):
        return self.__nim

    # Setter
    def set_nama(self, nama_baru):
        self.__nama = nama_baru

```

```

# ABSTRACT METHOD

@abstractmethod
def tampilkan_info(self):
    pass

# =====
# INHERITANCE
# =====

class MahasiswaInformatika(Mahasiswa, StateMachine):

    def __init__(self, nama, nim, bahasa):
        Mahasiswa.__init__(self, nama, nim)
        StateMachine.__init__(self)

        self.bahasa = Bahasa

# POLYMORPHISM
def tampilkan_info(self):
    print("===== MAHASISWA INFORMATIKA =====")
    print("Nama   :", self.get_nama())
    print("NIM    :", self.get_nim())
    print("Bahasa  :", self.bahasa)
    print("Status  :", self.state)
    print()

class MahasiswaSistemInformasi(Mahasiswa, StateMachine):

    def __init__(self, nama, nim, bidang):
        Mahasiswa.__init__(self, nama, nim)
        StateMachine.__init__(self)

        self.bidang = bidang

```

```

# POLYMORPHISM
def tampilkan_info(self):
    print("===== MAHASISWA SISTEM INFORMASI =====")
    print("Nama   :", self.get_nama())
    print("NIM    :", self.get_nim())
    print("Bidang  :", self.bidang)
    print("Status  :", self.state)
    print()
# =====
# CLASS DOSEN
# =====
class Dosen:
    def __init__(self, nama, nidn):
        self.nama = nama
        self.nidn = nidn

    def tampilkan_dosen(self):
        print("Nama Dosen :", self.nama)
        print("NIDN     :", self.nidn)
        print()

# =====
# CLASS MATA KULIAH
# =====
class MataKuliah:
    def __init__(self, kode, nama, sks):
        self.kode = kode
        self.nama = nama
        self.sks = sks

    def tampilkan_matkul(self):
        print("Kode MK :", self.kode)

```

```

        print("Nama MK :", self.nama)
        print("SKS    :", self.sks)
        print()

# =====
# CLASS NILAI
# =====

class Nilai:
    def __init__(self, tugas, uts, uas):
        self.tugas = tugas
        self.uts = uts
        self.uas = uas

    def hitung_nilai(self):
        total = (self.tugas + self.uts + self.uas) / 3
        return total

# =====
# CLASS KELAS
# =====

class Kelas:
    def __init__(self, nama_kelas):
        self.nama_kelas = nama_kelas
        self.daftar_mahasiswa = []

    # CRUD TAMBAH
    def tambah_mahasiswa(self, mahasiswa):
        self.daftar_mahasiswa.append(mahasiswa)
        print(f'{mahasiswa.get_nama()} berhasil ditambahkan')

    # CRUD HAPUS
    def hapus_mahasiswa(self, mahasiswa):
        self.daftar_mahasiswa.remove(mahasiswa)
        print(f'{mahasiswa.get_nama()} berhasil dihapus')

```



```

def tampilkan_semua(self):
    print(f"\n===== DAFTAR MAHASISWA {self.nama_kelas} =====")
    for mhs in self.daftar_mahasiswa:
        mhs.tampilkan_info()
# =====
# CLASS SISTEM KAMPUS
# =====
class SistemKampus:
    def __init__(self):
        self.daftar_dosen = []
        self.daftar_matkul = []

    def tambah_dosen(self, dosen):
        self.daftar_dosen.append(dosen)

    def tambah_matkul(self, matkul):
        self.daftar_matkul.append(matkul)

# =====
# OBJECT
# =====

# Mahasiswa
mhs1 = MahasiswaInformatika(
    "Andi",
    "231001",
    "Python"
)

mhs2 = MahasiswaSistemInformasi(
    "Siti",
    "231002",
    "Database"

```

```

)

# Dosen
dsn1 = Dosen(
    "Budi Santoso",
    "112233"
)

# Mata Kuliah
mk1 = MataKuliah(
    "IF101",
    "Pemrograman Python",
    3
)

# Nilai
nilai1 = Nilai(
    80,
    85,
    90
)

# Kelas
kelasA = Kelas("A")
# =====
# MENJALANKAN PROGRAM
# =====

# Login
mhs1.login()

# Tambah mahasiswa
kelasA.tambah_mahasiswa(mhs1)

```

```
kelasA.tambah_mahasiswa(mhs2)

# Tampilkan semua mahasiswa
kelasA.tampilkan_semua()

# State Machine
mhs1.change_state("Cuti")

# Kirim notifikasi
mhs1.kirim_notifikasi("Jadwal kuliah dimulai")

# Tampilkan dosen
dsn1.tampilkan_dosen()

# Tampilkan mata kuliah
mk1.tampilkan_matkul()

# Hitung nilai
print("Nilai Akhir :", nilai1.hitung_nilai())
```

BAB V

PENGUJIAN SISTEM

5.1 Tujuan Pengujian

Pengujian dilakukan untuk melihat apakah semua fitur yang ada pada Sistem Informasi Data Mahasiswa sudah berjalan dengan baik dan sesuai dengan yang diharapkan.

5.2 Pengujian Login

Skenario

Mencoba masuk ke dalam sistem menggunakan fitur login.

Kode

mhs1.login()

Hasil

Login berhasil

Status

Berhasil

5.3 Pengujian Menambah Mahasiswa

Skenario

Menambahkan data mahasiswa ke dalam kelas.

Kode

kelasA.tambah_mahasiswa(mhs1)

kelasA.tambah_mahasiswa(mhs2)

Hasil

```
Andi berhasil ditambahkan  
Siti berhasil ditambahkan
```

Status

Berhasil

5.4 Pengujian Menampilkan Data Mahasiswa

Skenario

Menampilkan data seluruh mahasiswa yang ada di dalam kelas.

Kode

```
kelasA.tampilkan_semua()
```

Hasil

```
===== MAHASISWA INFORMATIKA =====  
  
Nama : Andi  
  
NIM : 231001  
  
Bahasa : Python  
  
Status : Aktif  
  
===== MAHASISWA SISTEM INFORMASI =====  
  
Nama : Siti  
  
NIM : 231002  
  
Bidang : Database  
  
Status : Aktif
```

Status

Berhasil

5.5 Pengujian State Machine

Skenario

Mengubah status mahasiswa dari aktif menjadi cuti.

Kode

```
mhs1.change_state("Cuti")
```

Hasil

```
State berubah menjadi: Cuti
```

Status

Berhasil

5.6 Pengujian Notifikasi

Skenario

Mengirim pesan atau notifikasi kepada mahasiswa.

Kode

```
mhs1.kirim_notifikasi(  
    "Jadwal kuliah dimulai"  
)
```

Hasil

```
Notifikasi: Jadwal kuliah dimulai
```

Status

Berhasil

5.7 Pengujian Data Dosen

Skenario

Menampilkan informasi dosen yang tersimpan dalam sistem.

Kode

```
dsn1.tampilkan_dosen()
```

Hasil

Nama Dosen : Budi Santoso

NIDN : 112233

Status

Berhasil

5.8 Pengujian Data Mata Kuliah

Skenario

Menampilkan informasi mata kuliah yang ada di dalam sistem.

Kode

```
mk1.tampilkan_matkul()
```

Hasil

Kode MK : IF101

Nama MK : Pemrograman Python

SKS : 3

Status

Berhasil

5.9 Pengujian Perhitungan Nilai

Skenario

Menghitung nilai akhir mahasiswa berdasarkan nilai tugas, UTS, dan UAS.

Kode

```
nilai1 = Nilai(  
    80,  
    85,  
    90  
)  
nilai1.hitung_nilai()
```

Perhitungan

$$(80 + 85 + 90) / 3$$
$$= 255 / 3$$
$$= 85$$

Hasil

Nilai Akhir : 85.0

Status

Berhasil

5.10 Rekapitulasi Pengujian

Berdasarkan hasil pengujian yang sudah dilakukan, semua fitur pada sistem dapat digunakan dengan baik dan memberikan hasil sesuai yang diharapkan.

No	Pengujian	Hasil
1	Login	Berhasil
2	Tambah Mahasiswa	Berhasil
3	Tampilkan Mahasiswa	Berhasil

4	State Machine	Berhasil
5	Notifikasi	Berhasil
6	Data Dosen	Berhasil
7	Data Mata Kuliah	Berhasil
8	Perhitungan Nilai	Berhasil

Dari hasil pengujian tersebut, dapat dilihat bahwa seluruh fitur utama pada Sistem Informasi Data Mahasiswa sudah berjalan dengan baik. Mulai dari pengelolaan data mahasiswa, dosen, mata kuliah, hingga perhitungan nilai dan pengiriman notifikasi dapat dilakukan sesuai dengan kebutuhan sistem.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil pembuatan dan pengujian yang telah dilakukan, Sistem Informasi Data Mahasiswa berhasil dibuat menggunakan bahasa pemrograman Python dengan menerapkan konsep Object Oriented Programming (OOP). Sistem ini dapat digunakan untuk mengelola data mahasiswa, dosen, mata kuliah, serta nilai mahasiswa dengan cukup baik.

Selain itu, sistem juga memiliki beberapa fitur seperti login, pengiriman notifikasi, perubahan status mahasiswa, dan perhitungan nilai akhir mahasiswa. Dalam pembuatannya, sistem ini menerapkan beberapa konsep OOP, yaitu Abstraction, Encapsulation, Inheritance, Polymorphism, Mixins, dan State Machine. Dari hasil pengujian yang telah dilakukan, seluruh fitur dapat berjalan sesuai dengan fungsi yang diharapkan.

6.2 Saran

Walaupun sistem sudah dapat digunakan dengan baik, masih ada beberapa hal yang bisa ditambahkan agar sistem menjadi lebih lengkap dan lebih nyaman digunakan.

Beberapa pengembangan yang dapat dilakukan antara lain:

- 1) Menambahkan database seperti MySQL atau PostgreSQL agar data yang disimpan tidak hilang saat program ditutup.
- 2) Membuat tampilan antarmuka (GUI) supaya pengguna lebih mudah saat menjalankan sistem.
- 3) Menambahkan fitur pencarian data mahasiswa berdasarkan nama atau NIM.
- 4) Menambahkan fitur login menggunakan username dan password agar keamanan sistem lebih baik.
- 5) Mengembangkan sistem menjadi aplikasi berbasis web menggunakan Flask atau Django.

Dengan adanya pengembangan tersebut, sistem diharapkan dapat menjadi lebih lengkap, lebih praktis digunakan, dan lebih membantu dalam pengelolaan data akademik.

DAFTAR PUSTAKA

Deitel, P., & Deitel, H. (2019). Python for Programmers. Pearson.

docs.python.org

Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media.

Materi Perkuliahan Pemrograman Berorientasi Objek Menggunakan Python, UIN Sultan Syarif Kasim Riau.

Modul Praktikum Pemrograman Berorientasi Objek Menggunakan Python, UIN Sultan Syarif Kasim Riau.

Sommerville, I. (2016). Software Engineering (10th Edition). Pearson.